RECOVERY RUNBOOK

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

Fleet Management ↔ VCF Operations Recovery

Half-broken integration recovery, instance-limit deadlock, and Broadcom-aligned database cleanup procedure for VCF 9.0.1.

FLEET MANAGEMENT | VCF OPERATIONS | VRSLCM | INTEGRATION | RECOVERY | DB CLEANUP

VCF 9.0 VMware Cloud Foundation

VCF Operations ↔ Fleet Management Integration Recovery — RCA & Runbook

Prepared by: Virtual Control LLC **Date:** April 26, 2026 (rev. May 17, 2026) **Document Version:** 1.2 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0 **Fleet Management:** vmlcm-service-9.0.2.0-SNAPSHOT **VCF Operations:** 9.0.1.0.24960351

Where commands run in this RCA. Recovery touches:

- **Windows workstation (PowerShell / Bash with curl)** — REST API calls to Fleet Manager and VCF Operations. Token-based auth.
- **Fleet Manager / LCM appliance shell (SSH)** — appliance-only operations: `psql`, `service-control`, `vmon-cli`, `systemctl`, file edits in `/etc/`, `/var/lib/vrlcm/`, `/storage/`. Flagged "Run on the appliance only" — cannot run from workstation.
- **VCF Operations UI (browser)** — Fleet Management sub-section for click-driven workflow.

Per Option B, every workstation-runnable bash block has a PowerShell sibling. Replace any credential placeholder like `<your-password>` with values from your vault.

PowerShell users: Every bash block in this document that targets a REST API or other client-runnable command has a paste-ready PowerShell sibling block underneath it. For the full substitution table — `curl` → `Invoke-RestMethod`, base64 auth headers, here-docs → here-strings, `jq` / `python -m json.tool` → `ConvertFrom-Json` / `ConvertTo-Json -Depth N`, `plink` → `Invoke-SSHCommand` / `ssh.exe`, etc. — see POWERSHELL-TRANSLATION-REFERENCE.md. Blocks targeting the appliance shell only (`psql`, `service-control`, `vmon-cli`, `systemctl`, `journalctl`, file edits in `/etc/`, `/var/lib/vrlcm/`, `/storage/`) are flagged "Run on the appliance only" with no PS sibling — they cannot run from a Windows workstation.

Table of Contents

> **Related RCAs in this library** (`06-Troubleshooting/`): - `NSX-Manager-Cold-Start-Troubleshooting-Report` — NSX Manager appliance recovery (4 addenda: Corfu DB corruption, btrfs read-error after power-off, restore-from-backup path, post-restore stuck-state → fresh redeploy) - `VCF9-Management-Plane-Loss-RCA-2026-05-01` — vSAN object inaccessibility cascade after MM host rebuild with EA mode; source of the FDM-only rule for destructive host rebuilds.

1. Executive Summary

After an unplanned vcf-ops reboot during heavy vSAN resync activity, the **VCF Operations Fleet Management** integration entered a **half-broken state**:

- vcf-ops Admin UI **falsely reported** Fleet Management as **"Connected"**
- Fleet's logs continuously threw `vropsMasterNodeIP === (empty)` NPE
- Manifest retrieval failed in the UI ("*Retrieving 1 Manifest failed*")
- Disconnect → Reconnect failed with "*Error connection Fleet Management node, try again*"
- The standard Broadcom KB 402063 procedure **could not complete** because its DELETE workflow needs to log into vrops to clean up service accounts — and vrops was unreachable from Fleet because the address was null. **The bug blocked its own KB-prescribed fix.**

The actual blocker, surfaced from fleet's logs:

```
 Deployment or Import Blocked: You've reached the limit of allowed instances
for VCF Operations (maximum: 1). Please check your inventory and remove any
existing deployed, failed or draft instance(s) to proceed.
```

A stale `COMPLETED` -status environment in fleet's database consumed the only allowed `vrops` instance slot, blocking re-pair.

Resolution path (Broadcom-aligned, with snapshots as fallback):

1. Snapshot vcf-ops + fleet
2. Disconnect from vcops Admin UI
3. Enable temporary Fleet UI (`touch /var/lib/vrlcm/UI_ENABLED`) per KB 430561
4. Identify env IDs via Fleet Swagger `GET /lcm/lcops/api/v2/environments`

5. Delete orphan UI_INPROGRESS environments via Fleet API (`deleteFromVcenter:false`)
6. Direct SQL cleanup of stuck vrops env across the FK chain (9 tables, 83 rows)
7. Reconnect via vcops Admin UI — succeeds because instance slot is now free
8. Verify fresh env created with populated fields, NPE flood gone
9. Disable Fleet UI flag (`rm /var/lib/vrlcm/UI_ENABLED`)

Total elapsed time during incident: ~3 hours (most of it diagnostic). The recovery itself, with this runbook in hand, takes **under 30 minutes**.

---

2. Trigger Symptoms — When to Use This Runbook

Use this runbook when **two or more** of these symptoms are present:

Symptom set:

- vcf-ops Admin UI shows `Fleet Management: Connected` (green) AND
- Fleet's `/var/log/vrlcm/vmware_vrlcm.log` throws `VropsAddressException: Invalid Operations master address` and `NullPointerException` continuously (often hundreds per minute)
- VCF Operations UI shows "*VCF Operations Fleet Management Is Not Ready*"
- Manifest retrieval in deploy workflows fails with "*Retrieving 1 Manifest failed. A problem has occurred on the server. Please retry or contact the service provider and provide the reference token.*"
- Reconnect attempts fail with "*Error connection Fleet Management node, try again*"
- Fleet logs contain instance-limit error: "*You've reached the limit of allowed instances for VCF Operations (maximum: 1)*"

This runbook does NOT apply to: - True network/DNS failures (KB 410260) — fix DNS first - Cert expiration (KB 432320) — replace cert first - Password mismatch from CLI password change (KB 420284) — sync passwords first - Fresh install, never-connected state (KB 404388 §"For New Deployments") — redeploy fleet OVA

---

3. Environment Reference

Component	Host	IP	OS	Role
vcenter	esxi01	192.168.1.69	vCenter 9.0.2.0.25148086	Management
vcf-ops	esxi03	192.168.1.77	Photon OS 5	VCF Operations 9.0.1.0.24960351
fleet	esxi01	192.168.1.78	Photon OS 5	Fleet Management (vmlcm-service-9.0.2.0-SNAPSHOT)
sddc-manager	esxi02	192.168.1.241	—	SDDC Manager

Component	Host	IP	OS	Role
nsx-manager	esxi04	192.168.1.71	—	NSX 9.0.1

Authentication used in this runbook (verified):

Target	Username	Password
Fleet API (Basic auth)	`admin@local`	`<your-password>`
Fleet root SSH	`root`	`<your-password>`
vCenter	`administrator@vsphere.local`	`<your-password>`
vcf-ops root SSH	—	**Disabled** (use Invoke-VMScript via VMware Tools)

> **Note:** vcf-ops on VCF Operations 9.0.x ships with SSH disabled. Use PowerCLI's `Invoke-VMScript` to run guest commands when SSH is unavailable.

---

4. Root Cause Analysis

4.1 The Half-Broken Registration

In a healthy state, vcf-ops and fleet have a **bidirectional** registration:

- vcf-ops side stores: fleet appliance address, admin credential ref, cert
- fleet side stores: vROps cluster master address (`vropsMasterNodeIP`), service account refs

If either side's registration record is created while the other side is unhealthy or unreachable, the result is **asymmetric**: one side believes the registration is complete, the other has a row with null/empty critical fields.

In our incident: 1. vcf-ops was rebooted while `vmware-vcops` was still starting (5-min systemd `TimeoutStartSec` exceeded; service marked `failed`) 2. During this window, fleet's scheduled `SyncProxyFromVcfOpsTask` ran, found vrops unreachable, and *should* have left the integration record empty — but the DB row had already been created in a prior workflow 3. Result: fleet has an env row in `vm_lcops_environments` with `environmentStatus=COMPLETED` , but `vropsMasterNodeIP` is null (the field is set lazily by the sync task, which never succeeded) 4. Every minute thereafter, fleet's recurring sync tasks try to use the null field and throw NPE

vcops Admin UI's "Connected" status is just a check that fleet's appliance address responds — it does NOT validate that fleet's reverse-record back to vrops is populated. This is why the UI lies.

4.2 The Instance-Limit Deadlock

Fleet enforces a **hard limit of 1 instance per product type** (vrops, vra, vrli, vrni). The stale env row consumes that slot:

```
Deployment or Import Blocked: You've reached the limit of allowed instances
for VCF Operations (maximum: 1).
```

When you click **Connect** in vcops Admin UI to re-pair: - vcops sends fleet a register-environment request - Fleet rejects: instance limit hit - vcops surfaces: "*Error connection Fleet Management node, try again*" (KB 404388)

You cannot proceed without removing the stale slot occupier.

4.3 The Circular Delete Failure

Broadcom's KB-prescribed remedy (KB 402063) calls `DELETE /lcm/lcops/api/v2/environments/{id}/delete` to remove the stale env. The delete workflow includes a sub-task **`VropsDeleteCapabilitiesTask`** that:

1. Logs into vrops using the env record's stored address
2. Removes fleet's service accounts from vrops
3. Then proceeds with the local DB cleanup

But the env's stored address is **null** (the very bug we're fixing). The login fails with NPE. Even though `CONTINUE_ON_FAILURE: true` is set in the task spec, the overall workflow is marked `FAILED` and **the local DB row is NOT deleted**.

```
 ERROR VropsDeleteCapabilitiesTask -- Error while deleting capabilities from VCF Ops
      caused by NullPointerException
=> Updating Environment request status to FAILED
```

The circular dependency: the bug blocks the API path that exists to fix the bug. Broadcom's developer documentation for the DELETE API (verified: only 4 body fields — `controllerType`, `deleteFromInventory`, `deleteFromVcenter`, `deleteLbFromSddc`, `deleteWindowsVMs`) provides **no `force` or `skipCleanup` parameter** to bypass the failed sub-task.

The supported escape hatch per KB 404388 is "*Contact Broadcom Support (KB 410260) for manual database cleanup, as stale environment records in the node database may require API and database commands.*" In a homelab with snapshots, that translates to a documented direct-DB cleanup, following the same pattern Broadcom uses for stuck vApp records (KB 372992) and stuck NSX Transport Zones (KB 377552).

---

5. Diagnostic Procedure (Verbatim Commands)

5.1 Confirm vcf-ops Side Services

vcf-ops typically has SSH disabled. Use PowerCLI `Invoke-VMScript`:

```
 $pw = ConvertTo-SecureString '<your-password>' -AsPlainText -Force
$cred = New-Object System.Management.Automation.PSCredential('administrator@vsphere.local', $pw)
Connect-VIServer -Server 192.168.1.69 -Credential $cred -Force
$guest = New-Object System.Management.Automation.PSCredential('root', $pw)
$vm = Get-VM -Name 'vcf-ops'

Invoke-VMScript -VM $vm -GuestCredential $guest -ScriptType Bash -ScriptText @'
echo "=== is-system-running ==="
systemctl is-system-running
echo "=== Failed units ==="
systemctl --failed --no-pager --no-legend
echo "=== vmware-vcops status ==="
systemctl status vmware-vcops --no-pager -n 5
echo "=== suite-api response (must be 401, NOT 503) ==="
```

```
curl -k -s -o /dev/null -w "suite-api: %{http_code}\n" https://localhost/suite-api/api/versions
'@
```

Expected after fix: - `is-system-running`: `degraded` is OK if only `systemd-networkd-wait-online` is failed - `vmware-vcops`: `active` - `suite-api`: `401` (auth required = up; `503` means platform service is down)

If `vmware-vcops` is in failed state with `Result: timeout`, install the timeout drop-in:

```
# Run on the appliance only (vcf-ops Photon shell — systemd drop-in under /etc/systemd/system/, daemon-reload, reset-failed)
# No PowerShell sibling: file edits under /etc/ + systemctl cannot run from Windows.
# To deliver the same fix from a workstation, wrap this script body in PowerCLI Invoke-VMScript -ScriptType Bash against the vcf-ops VM (Connect-VIServer first; see Phase 1 snapshot block above for the credential pattern).
mkdir -p /etc/systemd/system/vmware-vcops.service.d
cat > /etc/systemd/system/vmware-vcops.service.d/timeout.conf <<EOF
[Service]
TimeoutStartSec=1200
EOF
systemctl daemon-reload
systemctl reset-failed vmware-vcops
systemctl start --no-block vmware-vcops
```

5.2 Confirm Fleet Side Services

Fleet allows SSH as `root`:

```
# Run on the appliance only (probes systemd unit state and listening sockets on fleet via SSH wrapper).
# No PowerShell sibling: the inner commands (systemctl, ss) are appliance-only — the plink call is just the transport.
# To run from Windows, replace `plink` with `ssh.exe` or PowerShell's `Invoke-SSHCommand` (Posh-SSH module) — the bash payload itself is unchanged.
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "systemctl is-active vrlcm-server vmtoolsd; \
   ss -tlnp | grep -E ':443|:8080'; \
   curl -k -s -o /dev/null -w 'health: %{http_code}\n' https://localhost/lcm/api/v1/health"
```

Expected: - `vrlcm-server`: `active` - `vmtoolsd`: `active` - Port 443 (nginx) and 8080 (Java backend) listening - `health: 200`

5.3 Search for Instance Limit Error

This is the smoking gun for the deadlock:

```
# Greps fleet's vrlcm log for the instance-limit signature (workstation -> fleet via plink SSH).
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "grep -E 'maximum.*1|instance limit|reached the limit' \
   /var/log/vrlcm/vmware_vrlcm.log | tail -3"
```

```
# PowerShell sibling - greps fleet's vrlcm log for the instance-limit signature.
# PS 5.1 cert-bypass header (paste once per session before any Invoke-RestMethod / Invoke-WebRequest call to a self-signed endpoint):
Add-Type @"
using System.Net;
using System.Security.Cryptography.X509Certificates;
public class TrustAll : ICertificatePolicy {
  public bool CheckValidationResult(ServicePoint sp, X509Certificate cert, WebRequest req, int problem) {
```

```
return true; }
}
"@
[System.Net.ServicePointManager]::CertificatePolicy = New-Object TrustAll
[System.Net.ServicePointManager]::SecurityProtocol = [System.Net.SecurityProtocolType]::Tls12

# Use ssh.exe (built into Win10/11) - bash payload unchanged. Quote the remote command with single quotes
on the PowerShell side, double quotes inside.
ssh.exe -o StrictHostKeyChecking=no root@192.168.1.78 'grep -E "maximum.*1|instance limit|reached the
limit" /var/log/vrlcm/vmware_vrlcm.log | tail -3'
```

If you see:

```
 InvalidEnvironmentException: Deployment or Import Blocked: You've reached
the limit of allowed instances for VCF Operations (maximum: 1).
```

→ This runbook applies. Proceed.

5.4 Confirm Network/DNS Not the Cause

Rule out the easier KB-410260 path before touching the DB:

```
 # DNS + reachability probe FROM fleet TO vcf-ops (workstation -> fleet via plink, then
resolv.conf/nslookup/curl run on fleet).
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "cat /etc/resolv.conf; \
   nslookup vcf-ops.lab.local; \
   curl -k -s -o /dev/null -w 'vcf-ops via IP: %{http_code}\n' https://192.168.1.77/; \
   curl -k -s -o /dev/null -w 'vcf-ops via FQDN: %{http_code}\n' https://vcf-ops.lab.local/"
```

```
 # PowerShell sibling - DNS+reachability probe. The resolv.conf / nslookup view from FLEET still requires
SSH (left as ssh.exe).
# The HTTPS reachability probes can be done directly from Windows (different vantage point - good cross-
check).
ssh.exe -o StrictHostKeyChecking=no root@192.168.1.78 'cat /etc/resolv.conf; nslookup vcf-ops.lab.local'

# Resolve FQDN from Windows DNS:
Resolve-DnsName -Name 'vcf-ops.lab.local' -ErrorAction Stop

# HTTPS reachability from the Windows workstation (cert-bypass header above must be in scope):
foreach ($u in @('https://192.168.1.77/','https://vcf-ops.lab.local/')) {
  try {
    $r = Invoke-WebRequest -Uri $u -UseBasicParsing -TimeoutSec 10 -ErrorAction Stop
    "{0,-40} HTTP {1}" -f $u, $r.StatusCode
  } catch {
    "{0,-40} ERROR {1}" -f $u, $_.Exception.Message
  }
}
```

Expected: - `nameserver` configured - `nslookup` resolves the vcops FQDN - Both curl probes return `200`

If any of these fail, fix DNS/network first per KB 410260 — do NOT proceed to DB cleanup.

---

6. Recovery Procedure (Phase by Phase)

Total expected duration: ~30 minutes **Reversibility:** Snapshots taken in Phase 1 provide complete rollback **User-side actions:** Phases 2 (Disconnect) and 6 (Reconnect) are UI clicks; the rest are SSH/PowerCLI

6.1 Phase 1: Snapshots (Mandatory)

Do NOT proceed without these snapshots. Phase 5 writes to fleet's postgres in a transaction. If the FK chain identification is incomplete (a foreign-key constraint we missed will abort the transaction, but if the cleanup commits with orphan refs left, fleet may behave subtly wrong). Snapshots are the rollback path.

```
$pw = ConvertTo-SecureString '<your-password>' -AsPlainText -Force
$cred = New-Object System.Management.Automation.PSCredential('administrator@vsphere.local', $pw)
Connect-VIServer -Server 192.168.1.69 -Credential $cred -Force
$ts = (Get-Date -Format 'yyyyMMdd-HHmmss')
$tag = "pre-fleet-disconnect-$ts"
foreach ($n in 'vcf-ops','fleet') {
  Get-VM -Name $n | New-Snapshot -Name $tag `
    -Description "Pre Fleet disconnect/reconnect per KB 402063 - $ts" `
    -Memory:$false -Quiesce:$false -Confirm:$false
}
```

6.2 Phase 2: Disconnect via vcf-ops Admin UI

1. Open `https://<vcf-ops>/admin` (e.g. `https://192.168.1.77/admin`)
2. **System Status** page → **Fleet Management** card
3. Click **Disconnect**
4. Confirm

The Admin UI may report success quickly even though the back-end disconnect is incomplete. That's fine — we'll forcibly clean the fleet side in Phase 5.

6.3 Phase 3: Enable Fleet UI and Find Env IDs

The vcops Developer Center's **Fleet Management API** card hides itself when fleet is disconnected (the UI proxies through the integration). To access fleet's Swagger directly, enable Fleet's own UI flag (per KB 430561):

```
# Run on the appliance only (writes a file under /var/lib/vrlcm/ on fleet - skip-rule path).
# No PowerShell sibling: file edits under /var/lib/vrlcm/ cannot be made from a Windows workstation.
# To deliver from a workstation: ssh.exe to fleet and run the same commands (plink -> ssh.exe is the only change).
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "touch /var/lib/vrlcm/UI_ENABLED && ls -la /var/lib/vrlcm/UI_ENABLED"
```

Open Fleet Swagger directly:

`https://192.168.1.78/api/swagger-ui/index.html`

> If that URL doesn't render, try `https://192.168.1.78/lcm/api/swagger-ui.html` or `/lcm/swagger-ui/index.html`.

In Swagger:

1. Section: **Environment Controller**
2. Endpoint: **`GET /lcm/lcops/api/v2/environments`**
3. Click **Try it out → Execute**
4. Capture the response — note each env's `environmentId`, `environmentStatus`, and `products[].id`

Identify the three categories:

Category	Identifying field	Action in this runbook
vrops integration (the broken one)	`products[0].id == "vrops"` AND `environmentStatus == "COMPLETED"`	Phase 5 (DB cleanup)
Orphan deploy attempt	`products[0].id` ∈ { `vra`, `vrli`, `vrni` } AND `environmentStatus == "UI_INPROGRESS"`	Phase 4 (API delete)
Healthy deployment	`products[0].id` ∈ { `vra`, `vrli`, `vrni` } AND `environmentStatus == "COMPLETED"`	LEAVE ALONE

If you also need the API directly via curl (basic auth):

```
# Lists every Fleet environment (vrops, vra, vrli, vrni) with products/nodes/properties - identifies
which env occupies the instance slot.
curl -k -s -u 'admin@local:<your-password>' \
  https://192.168.1.78/lcm/lcops/api/v2/environments | python -m json.tool
```

```
# PowerShell sibling - lists Fleet environments via Invoke-RestMethod. Cert-bypass header (added at first
PS sibling above) must be in scope.
$pair    = 'admin@local:<your-password>'
$basic   = 'Basic ' + [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($pair))
$headers = @{ Authorization = $basic; Accept = 'application/json' }

$envs = Invoke-RestMethod -Method GET -Uri 'https://192.168.1.78/lcm/lcops/api/v2/environments' -Headers
$headers
$envs | ConvertTo-Json -Depth 8

# Quick categorization to feed Phase 4 / Phase 5 decisions:
$envs | ForEach-Object {
  [pscustomobject]@{
    EnvId    = $_.environmentId
    Status   = $_.environmentStatus
    Products = ($_.products.id) -join ','
  }
} | Format-Table -AutoSize
```

6.4 Phase 4: Delete Clean Orphans via API

Orphan UI_INPROGRESS envs (failed deploy attempts) have no live vrops capability cleanup, so the API DELETE works cleanly on them. Run this for each orphan env ID:

```
# Deletes ONE orphan environment in Fleet. deleteFromVcenter MUST be false - see danger-box below.
ENV_ID="<orphan-env-uuid>"
curl -k -s -u 'admin@local:<your-password>' \
  -X DELETE -H 'Content-Type: application/json' -H 'Accept: application/json' \
  -d
'{"controllerType":"string","deleteFromInventory":true,"deleteFromVcenter":false,"deleteLbFromSddc":true,"
deleteWindowsVMs":true}' \
  -w '\nHTTP %{http_code}\n' \
  "https://192.168.1.78/lcm/lcops/api/v2/environments/$ENV_ID"
```

```
# PowerShell sibling - deletes ONE orphan Fleet environment via Invoke-RestMethod. deleteFromVcenter MUST
be $false.
$EnvId   = '<orphan-env-uuid>'
$pair    = 'admin@local:<your-password>'
$basic   = 'Basic ' + [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($pair))
$headers = @{ Authorization = $basic; 'Content-Type' = 'application/json'; Accept = 'application/json' }

$body = @{
  controllerType      = 'string'
  deleteFromInventory = $true
  deleteFromVcenter   = $false # CRITICAL - $true would remove component VMs from vCenter
  deleteLbFromSddc    = $true
  deleteWindowsVMs    = $true
} | ConvertTo-Json -Depth 4

try {
  $resp = Invoke-RestMethod -Method DELETE `
            -Uri ('https://192.168.1.78/lcm/lcops/api/v2/environments/{0}' -f $EnvId) `
            -Headers $headers -Body $body
  $resp | ConvertTo-Json -Depth 6
} catch {
  Write-Warning ("DELETE failed: {0}" -f $_.Exception.Message)
  if ($_.Exception.Response) { Write-Warning ("HTTP {0}" -f [int]$_.Exception.Response.StatusCode) }
}
```

`deleteFromVcenter` **MUST be** `false`. Setting it to `true` instructs fleet to remove component VMs from vCenter. If applied to the live `vrops` env, **this would delete the vcf-ops VM itself**. The Swagger UI's "Try it out" auto-fills the body with all-true defaults — you must edit `deleteFromVcenter: true` → `false` before Execute.

Verify each orphan completed successfully:

```
# Greps fleet's log for DELETE_ENVIRONMENT request status transitions (workstation -> fleet via plink).
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "tail -300 /var/log/vrlcm/vmware_vrlcm.log | grep 'request status to.*DELETE_ENVIRONMENT' | tail -5"
```

```
# PowerShell sibling - same log grep via ssh.exe.
ssh.exe -o StrictHostKeyChecking=no root@192.168.1.78 'tail -300 /var/log/vrlcm/vmware_vrlcm.log | grep
"request status to.*DELETE_ENVIRONMENT" | tail -5'
```

Look for `Updating the Environment request status to COMPLETED` for each request ID.

6.5 Phase 5: Database Cleanup of Stuck vrops Env

This is the heart of the runbook. Once orphans are gone, only the broken `vrops` env remains. Its API DELETE will fail (the circular issue). Direct DB cleanup is required.

6.5.1 Discover the dependency chain

Foreign key constraints in fleet's `vrlcm` postgres (verified for 9.0.x):

```
vm_lcops_environments (vmid, PK)
 ├── vm_lcops_infrastructure_properties (environment_vmid)  FK
 └── vm_lcops_product (environment_vmid)                    FK
       ├── vm_lcops_product_nodes (product_vmid)            FK
       │     └── vm_lcops_node_properties (node_vmid)       FK
       └── vm_lcops_product_properties (product_vmid)       FK
```

Plus ancillary tables (no FK, but reference the env by string):

- `vm_engine_scheduledrequest` — recurring sync tasks; **deleting these stops the NPE flood**
- `vm_locker_reference` — credential refs scoped to env
- `vm_locker_certificate` — per-env certs (alias contains env UUID)

Audit/history tables that contain the env UUID embedded in JSON columns but are NOT load-bearing:

- `vm_engine_event`, `vm_engine_execution_request`, `vm_engine_user_request`, `vm_rs_request`

Leave the audit tables alone — they are append-only history; the env UUID appearing inside JSON blobs causes no harm and removing them strips audit trail without benefit.

6.5.2 Identify the three UUIDs

You need three UUIDs from the GET response in Phase 3:

Symbol	Where in JSON
`$ENV`	Top-level `environmentId`
`$PROD`	`products[0].vmid`
`$NODE`	`products[0].nodes[0].vmid`

Example from our incident: - `ENV = df6d02bb-692a-4c44-a0d3-99e29c672bd0` - `PROD = c386a73e-5f54-4425-9cbb-00233b00714d` - `NODE = ed918080-c493-4232-a880-3df0379b32df`

6.5.3 Run the cleanup transaction

See [Appendix A](#) for the full reusable SQL with parameter substitution. Brief form:

```
# Run on the appliance only (writes /tmp/cleanup.sql on fleet, then runs psql against the local vrlcm database).
# No PowerShell sibling: psql against fleet's embedded vpostgres only listens on 127.0.0.1 - cannot run from a Windows workstation.
# To deliver from a workstation: ssh.exe to fleet and run the same heredoc + psql invocation (replace plink with ssh.exe; payload unchanged).
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  'cat > /tmp/cleanup.sql <<SQL
\set ON_ERROR_STOP on
\set ENV  '"'"'<env-uuid>'"'"'
\set PROD '"'"'<prod-uuid>'"'"'
\set NODE '"'"'<node-uuid>'"'"'
```

```
BEGIN;
DELETE FROM vm_engine_scheduledrequest WHERE vmid LIKE '"'"'%<env-uuid>%'"'"' OR targetid LIKE '"'"'%<env-uuid>%'"'"';
DELETE FROM vm_locker_reference WHERE envid = :'"'"'ENV'"'"';
DELETE FROM vm_locker_certificate WHERE alias LIKE '"'"'%<env-uuid>%'"'"';
DELETE FROM vm_lcops_node_properties WHERE node_vmid = :'"'"'NODE'"'"';
DELETE FROM vm_lcops_product_nodes WHERE product_vmid = :'"'"'PROD'"'"';
DELETE FROM vm_lcops_product_properties WHERE product_vmid = :'"'"'PROD'"'"';
DELETE FROM vm_lcops_product WHERE environment_vmid = :'"'"'ENV'"'"';
DELETE FROM vm_lcops_infrastructure_properties WHERE environment_vmid = :'"'"'ENV'"'"';
DELETE FROM vm_lcops_environments WHERE vmid = :'"'"'ENV'"'"';
SELECT count(*) AS env_remaining FROM vm_lcops_environments WHERE vmid = :'"'"'ENV'"'"';
COMMIT;
SQL
PGPASSWORD=vmware /opt/vmware/vpostgres/current/bin/psql -h 127.0.0.1 -p 5432 -U postgres -d vrlcm -f /tmp/cleanup.sql'
```

Expected output (our incident, totaling 83 rows across 9 tables):

```
 DELETE 2     -- vm_engine_scheduledrequest
DELETE 3     -- vm_locker_reference
DELETE 1     -- vm_locker_certificate
DELETE 30    -- vm_lcops_node_properties
DELETE 1     -- vm_lcops_product_nodes
DELETE 20    -- vm_lcops_product_properties
DELETE 1     -- vm_lcops_product
DELETE 24    -- vm_lcops_infrastructure_properties
DELETE 1     -- vm_lcops_environments
env_remaining = 0
COMMIT
```

`\set ON_ERROR_STOP on` **is critical.** It causes psql to abort on the first error and leaves the transaction in an aborted state, so `COMMIT` becomes an implicit `ROLLBACK`. Without this, a partial cleanup could commit. In our incident's first attempt, this safety net saved us when we missed `vm_lcops_product_nodes` in the dependency chain — the transaction rolled back cleanly, no data was lost, and we re-ran with the corrected SQL.

6.5.4 Verify NPE flood stopped

```
 # Counts ERROR lines in fleet's vrlcm log within the last 60 seconds (workstation -> fleet via plink).
Expected post-cleanup: 0.
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "CUTOFF=\$(date -u -d '60 seconds ago' '+%Y-%m-%dT%H:%M:%S'); \
   awk -v c=\"\$CUTOFF\" '\$1 >= c' /var/log/vrlcm/vmware_vrlcm.log | grep -cE 'ERROR'"
```

```
 # PowerShell sibling - counts ERROR lines in fleet's vrlcm log within the last 60s. Same logic, single-quoted on the PS side so $vars are passed literally to bash.
ssh.exe -o StrictHostKeyChecking=no root@192.168.1.78 'CUTOFF=$(date -u -d "60 seconds ago" "+%Y-%m-%dT%H:%M:%S"); awk -v c="$CUTOFF" "\$1 >= c" /var/log/vrlcm/vmware_vrlcm.log | grep -cE "ERROR"'
```

Expected: `0` — within seconds of deleting `vm_engine_scheduledrequest` rows, the NPE flood stops because the recurring tasks themselves are gone.

6.6 Phase 6: Reconnect via vcf-ops Admin UI

Back in `https://<vcf-ops>/admin`:

1. **System Status → Fleet Management → Connect**
2. **Step 1 — Node Details:** - Node Address: `fleet.lab.local` (FQDN preferred over IP) - Admin Password: standard `<your-password>` - Click **Test Connection** - Cert dialog shows fleet's TLS cert → **Accept Certificate → Next**
3. **Step 2 — Password:** whatever the dialog labels it (typically the cluster bootstrap password or vCenter admin)
4. Click **Submit/Connect**

Expected outcome: wizard completes; vcops Admin UI shows Fleet Management as `Connected` with no banner; a **fresh** env record appears in fleet's DB with a **new** `environmentId` (different from the one you just deleted).

6.7 Phase 7: Verify and Cleanup

Verify fresh env was created

```
 # Lists Fleet environments and prints (id, status, products) - confirms a fresh vrops env exists with a
NEW UUID.
curl -k -s -u 'admin@local:<your-password>' \
  https://192.168.1.78/lcm/lcops/api/v2/environments \
  | python -c "import sys,json; d=json.load(sys.stdin); [print(e['environmentId'], e['environmentStatus'],
[p['id'] for p in e.get('products',[])]) for e in d]"
```

```
 # PowerShell sibling - same listing via Invoke-RestMethod. Cert-bypass header must be in scope.
$pair    = 'admin@local:<your-password>'
$basic   = 'Basic ' + [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes($pair))
$headers = @{ Authorization = $basic; Accept = 'application/json' }

Invoke-RestMethod -Method GET -Uri 'https://192.168.1.78/lcm/lcops/api/v2/environments' -Headers $headers
|
  ForEach-Object {
    "{0}  {1}  [{2}]" -f $_.environmentId, $_.environmentStatus, (($_.products.id) -join ',')
  }
```

Expected: one env, status `COMPLETED`, products `['vrops']`, ID **different** from what you deleted.

Verify bootstrap success in fleet log

```
 # Greps fleet's vrlcm log for the post-reconnect bootstrap SUCCESS marker.
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "grep 'bootstrap.*SUCCESS' /var/log/vrlcm/vmware_vrlcm.log | tail -3"
```

```
 # PowerShell sibling - same grep via ssh.exe.
ssh.exe -o StrictHostKeyChecking=no root@192.168.1.78 'grep "bootstrap.*SUCCESS"
/var/log/vrlcm/vmware_vrlcm.log | tail -3'
```

Expected: `Result: [bootstrap ------------- SUCCESS`

Disable the temporary Fleet UI

Per KB 430561:

```
 # Run on the appliance only (removes the flag file under /var/lib/vrlcm/ on fleet - skip-rule path).
# No PowerShell sibling: file deletes under /var/lib/vrlcm/ cannot be made from a Windows workstation.
# To deliver from a workstation: ssh.exe to fleet and run the same rm (plink -> ssh.exe is the only
change).
```

```
plink -ssh -batch -pw '<your-password>' root@192.168.1.78 \
  "rm -v /var/lib/vrlcm/UI_ENABLED"
```

Confirm vcops UI side

In vcf-ops UI (not admin) — VCF Operations Fleet Management widgets/cards should now load without the "*Not Ready*" banner. Manifest retrieval (the original symptom) should succeed.

---

7. Critical Safety Constraints

Rule	Why	What goes wrong if violated
`deleteFromVcenter: false` in every API delete payload	The body's default in Swagger is all-true	Live VMs get deleted from vCenter
Snapshots before Phase 5	DB writes are not transparently reversible	If FK chain analysis is wrong, no rollback
`\set ON_ERROR_STOP on` in psql script	Single-statement failure must abort the transaction	Partial commit leaves orphan refs
Children-first order in DELETE	FK constraints require it	Transaction aborts (caught by ON_ERROR_STOP)
Run on the **vrlcm** DB, not `postgres`	Wrong DB has no relevant tables	Commands silently no-op, you think it worked
Verify `env_remaining = 0` BEFORE COMMIT	The COMMIT is final	A failed mid-DELETE commits a partial cleanup
Use FQDN for the Connect step (not IP)	KB 430561 — `/casa/cluster/status` returns IP that may not match cert SAN	"Internal server error" on Connect

---

8. Verification Checklist

After Phase 7, all of these should be true:

- [ ] vcf-ops Admin UI shows `Fleet Management: Connected` (green)
- [ ] Fleet UI flag removed (`/var/lib/vrlcm/UI_ENABLED` does not exist)
- [ ] Fleet's `vrlcm-server` log has zero `vropsMasterNodeIP` / `VropsAddressException` entries in the last 60s
- [ ] Fleet's env table has exactly one row, with a **new** `environmentId` (not the one you deleted), `environmentStatus = COMPLETED`
- [ ] Fleet's bootstrap log shows `[bootstrap ------------- SUCCESS`
- [ ] vcf-ops VM is still PoweredOn at its expected IP (snapshot-equivalent state)
- [ ] vcf-ops `vmware-vcops` service is `active`
- [ ] vcf-ops `suite-api` returns HTTP `401` (not `503`)
- [ ] VCF Operations UI does NOT show "Fleet Management Not Ready"
- [ ] Manifest retrieval workflow does NOT show "Retrieving 1 Manifest failed"
- [ ] vcf-ops + fleet snapshots still exist (delete after 24-48h confidence period)

9. Rollback Plan

If at any phase something goes wrong:

1. **Phase 1–4 failures:** no DB writes yet — abort and reassess. No rollback needed.
2. **Phase 5 transaction error (uncommitted):** `\set ON_ERROR_STOP on` causes auto-ROLLBACK. Fix the SQL and retry.
3. **Phase 5 committed but Phase 6 reconnect still fails:** revert the fleet snapshot:

```
$snap = Get-VM -Name 'fleet' | Get-Snapshot -Name 'pre-fleet-disconnect-*'
Set-VM -VM (Get-VM -Name 'fleet') -Snapshot $snap -Confirm:$false
```

After revert, fleet is back to the broken-but-known state. From there, either retry the procedure with corrected analysis or open a Broadcom support case (KB 410260).

4. **Catastrophic outcome (vcops or fleet won't boot):** revert both snapshots. Worst-case: redeploy fleet OVA per KB 404388 §"For New Deployments".

10. Broadcom KB Cross-References

KB	Title	Relevance
402063	Troubleshooting VCF Operations fleet management node Integration Failures	Primary procedure (Disconnect → Get Env ID → Delete → Reconnect)
404388	"Error connection Fleet management node, try again"	Exact error text seen on Reconnect; recommends Support case for persistent failures
410260	Same error message — DNS / instance limit context	Identifies the "maximum: 1 instances" diagnostic
430561	Reconnecting Fleet Management with VCF Operation fails — "Internal server error"	Source of the `touch /var/lib/vrlcm/UI_ENABLED` workaround
420284	"VCF Operations Fleet Management Is Not Ready"	NOT applicable to this scenario (covers password mismatch from CLI password change)
432320	Fleet Management not connected after SSL cert change	Rule out before this runbook
414992	Fleet Management loses networking on reboot/upgrade to 9.0.1	Rule out before this runbook
420856	"Error pushing service registry"	Possible during Phase 6 reconnect — separate workaround
417122	Unable to integrate Fleet Management with VCF Operations	General reference
372992	Unable to delete a vApp in Unresolved/Failed state	Documented Broadcom pattern: DB cleanup when API DELETE fails on stuck record

KB	Title	Relevance
377552	Stale Transport Zone Profiles fail to delete with NullPointerException	Same DB-cleanup pattern in NSX

Broadcom Developer reference: - [Delete Environment V2 API schema]() — confirms only 4 body fields, no `force` / `skipCleanup` parameter

---

11. Appendix A — Full SQL Cleanup Script

This is the complete, reusable cleanup SQL with placeholder substitution. Save it as `/tmp/fleet-env-cleanup.sql` and run via `psql -f`.

```
 -- /tmp/fleet-env-cleanup.sql
-- Direct DB cleanup of a stuck VCF Operations / Fleet Management env record.
-- Pre-req: matching env-vmid, product-vmid, node-vmid from
--          GET /lcm/lcops/api/v2/environments

\set ON_ERROR_STOP on
\set ENV  'REPLACE-WITH-ENV-UUID'
\set PROD 'REPLACE-WITH-PROD-UUID'
\set NODE 'REPLACE-WITH-NODE-UUID'

\echo '=== PRE-CLEANUP COUNTS ==='
SELECT 'vm_lcops_environments' AS tbl, count(*) FROM vm_lcops_environments WHERE vmid = :'ENV'
UNION ALL SELECT 'vm_lcops_infrastructure_properties', count(*) FROM vm_lcops_infrastructure_properties
WHERE environment_vmid = :'ENV'
UNION ALL SELECT 'vm_lcops_product', count(*) FROM vm_lcops_product WHERE environment_vmid = :'ENV'
UNION ALL SELECT 'vm_lcops_product_nodes', count(*) FROM vm_lcops_product_nodes WHERE product_vmid =
:'PROD'
UNION ALL SELECT 'vm_lcops_node_properties', count(*) FROM vm_lcops_node_properties WHERE node_vmid =
:'NODE'
UNION ALL SELECT 'vm_lcops_product_properties', count(*) FROM vm_lcops_product_properties WHERE
product_vmid = :'PROD'
UNION ALL SELECT 'vm_locker_reference', count(*) FROM vm_locker_reference WHERE envid = :'ENV'
UNION ALL SELECT 'vm_locker_certificate', count(*) FROM vm_locker_certificate WHERE alias LIKE '%' ||
:'ENV' || '%'
UNION ALL SELECT 'vm_engine_scheduledrequest', count(*) FROM vm_engine_scheduledrequest WHERE vmid LIKE
'%' || :'ENV' || '%' OR targetid LIKE '%' || :'ENV' || '%';

\echo '=== BEGIN TRANSACTION ==='
BEGIN;

\echo '1. scheduled requests (stops NPE flood)'
DELETE FROM vm_engine_scheduledrequest
  WHERE vmid LIKE '%' || :'ENV' || '%'
     OR targetid LIKE '%' || :'ENV' || '%';

\echo '2. locker references and certs'
DELETE FROM vm_locker_reference   WHERE envid = :'ENV';
DELETE FROM vm_locker_certificate WHERE alias LIKE '%' || :'ENV' || '%';

\echo '3. node_properties (FK leaf)'
DELETE FROM vm_lcops_node_properties WHERE node_vmid = :'NODE';

\echo '4. product_nodes'
DELETE FROM vm_lcops_product_nodes WHERE product_vmid = :'PROD';

\echo '5. product_properties (by product_vmid - all rows)'
DELETE FROM vm_lcops_product_properties WHERE product_vmid = :'PROD';

\echo '6. product (FK to env)'
```

```
DELETE FROM vm_lcops_product WHERE environment_vmid = :'ENV';

\echo '7. infrastructure_properties (FK to env)'
DELETE FROM vm_lcops_infrastructure_properties WHERE environment_vmid = :'ENV';

\echo '8. environments (root row consuming the instance slot)'
DELETE FROM vm_lcops_environments WHERE vmid = :'ENV';

\echo '9. verify - all three core tables MUST show 0'
SELECT 'vm_lcops_environments' AS tbl, count(*) FROM vm_lcops_environments WHERE vmid = :'ENV'
UNION ALL SELECT 'vm_lcops_product', count(*) FROM vm_lcops_product WHERE vmid = :'PROD'
UNION ALL SELECT 'vm_lcops_product_nodes', count(*) FROM vm_lcops_product_nodes WHERE vmid = :'NODE';

\echo '=== COMMIT ==='
COMMIT;

\echo '=== POST-CLEANUP env table ==='
SELECT environmentid, status FROM vm_lcops_environments;
```

Run as:

```
 # Run on the appliance only (psql against fleet's embedded vpostgres on 127.0.0.1:5432).
# No PowerShell sibling: vpostgres only listens on loopback - cannot be reached from a Windows workstation.
PGPASSWORD=vmware /opt/vmware/vpostgres/current/bin/psql \
  -h 127.0.0.1 -p 5432 -U postgres -d vrlcm \
  -f /tmp/fleet-env-cleanup.sql
```

---

12. Appendix B — Fleet API Reference

Authentication

Two paths discovered during the incident:

Path	Status	Use case
`POST /api/v2/login` (JSON body `{username,password}`)	**Returns 401** even with valid credentials — body schema differs from public docs	Avoid
HTTP Basic Auth on the API endpoints directly	**Works** with `admin@local:<password>`	**Use this**

Key endpoints

Method	Path	Purpose
GET	`/lcm/lcops/api/v2/environments`	List all environments (returns full JSON with products/nodes/properties)
DELETE	`/lcm/lcops/api/v2/environments/{environmentId}`	Delete env (body required — see safety constraint below)
GET	`/lcm/api/v1/health`	Liveness check (no auth required, returns 200)

DELETE body schema (verified from Broadcom developer page)

```
{
  "controllerType": "string",
  "deleteFromInventory": true,
  "deleteFromVcenter": false,
  "deleteLbFromSddc": true,
  "deleteWindowsVMs": true
}
```

No additional parameters exist (no `force`, `skipCleanup`, `ignoreErrors`).

Swagger UI URLs (when `/var/lib/vrlcm/UI_ENABLED` is touched)

- `https://<fleet>/api/swagger-ui/index.html` — primary (verified during incident)
- `https://<fleet>/lcm/api/swagger-ui.html` — alt
- `https://<fleet>/lcm/swagger-ui/index.html` — alt

---

13. Appendix C — FK Tree and Table Reference

Foreign-key relationships in fleet's `vrlcm` database

```
vm_lcops_environments (vmid PK)
  ├── vm_lcops_infrastructure_properties (environment_vmid FK)
  │     constraint: fk1cveexgct0r5pxc3jlbiq5stw
  └── vm_lcops_product (environment_vmid FK, vmid PK)
        constraint: fkqwyrmo216nd2kyk0d4x1ixr1y
        ├── vm_lcops_product_nodes (product_vmid FK, vmid PK)
        │     constraint: fkto4vdx44wv2s30pktd4k0volu
        │     └── vm_lcops_node_properties (node_vmid FK)
        │           constraint: fkfohfj4lg6gt0dfinogrxlaxqr
        └── vm_lcops_product_properties (product_vmid FK)
              constraint: fkeo62jt4byet55y97mmigvxf2j
```

Tables touched by env-uuid string lookup (no FK)

Table	Column searched	Action
`vm_engine_scheduledrequest`	`vmid`, `targetid` (LIKE)	DELETE — stops NPE flood
`vm_locker_reference`	`envid` (=)	DELETE
`vm_locker_certificate`	`alias` (LIKE)	DELETE

Tables to LEAVE UNTOUCHED (audit history with embedded JSON references)

Table	Note
`vm_engine_event`	Append-only event log; substring matches are inside JSON `eventargument`
`vm_engine_execution_request`	Finished execution history
`vm_engine_user_request`	Historic user request log
`vm_rs_request`	Request system audit

Postgres connection details

	Value
Host	`127.0.0.1` (TCP) — no Unix socket exposed
Port	`5432`
Superuser	`postgres` / `vmware`
App user/DB	`vrlcm` / `vrlcm`
Binary	`/opt/vmware/vpostgres/current/bin/psql`

---

14. Lessons Learned

1. The vcops Admin UI status is NOT a source of truth for integration health. It can show "Connected" while the back-end is half-broken. Always corroborate with fleet-side log inspection (`grep VropsAddressException /var/log/vrlcm/vmware_vrlcm.log`).

2. Fleet's instance-limit enforcement is invisible until you hit it. There is no UI surface that warns you that a stuck record is consuming the single allowed slot. The error is only visible in fleet's log.

3. Broadcom's KB-prescribed API DELETE has a circular dependency. When the bug we're fixing is `vropsMasterNodeIP=null` , the DELETE workflow's `VropsDeleteCapabilitiesTask` cannot complete. There is no API parameter to bypass this. Direct DB cleanup is the documented escape hatch.

4. Snapshot before any Phase-5 DB write. The first cleanup attempt missed `vm_lcops_product_nodes` in the FK chain. The transaction aborted cleanly thanks to `ON_ERROR_STOP` , but if it had committed partially, snapshots would have been the only path back.

5. Swagger's "Try it out" auto-fills request bodies with all-true defaults. For DELETE operations, this is a silent foot-gun. Always edit `deleteFromVcenter: true` → `false` before clicking Execute.

6. `vcops` does not have a "cassandra" service in 9.x. It uses Gemfire + Postgres. Diagnostic scripts that probe `vmware-vcops-cassandra` will return `inactive` ; that's normal, not a fault.

7. vcf-ops `vmware-vcops` service can fail at boot with `Result: timeout` after heavy I/O events. The fix is a systemd drop-in (`TimeoutStartSec=1200`), not a redeploy. Check for this BEFORE concluding the Operations cluster is broken.

8. Cross-reference the symptom set carefully — multiple KBs share the same error message. "Error connection Fleet Management node, try again" appears in KB 404388, KB 410260, and KB 432320, each with different root causes (stale env vs DNS vs cert). Use fleet's logs to disambiguate.

9. KB 402063 is the public companion of the restricted KB 410260. When a recovery procedure is referenced inside a support-only KB, search for its public sibling before opening a case. KB 402063 contains the exact DELETE endpoint and payload (verbatim in §15 below) and was sufficient to resolve the 2026-05-17 recurrence without engaging Broadcom Support.

10. VCF 9.0.1 Fleet UI's `Components → Delete` is greyed out by design when the registration is the only/last 1:1 link. There is no UI force-delete in this build — go straight to the API (§15). KB 430561's "3-dots → Delete" wording targets pre-9.0.1 vRLCM and is misleading for the 9.0.1 build.

11. Quiesced snapshots can deadlock on K8s/Docker-heavy guests. Fleet's `kind-bootstrap-control-plane` container holds many open files; VMware Tools' guest-quiesce hook hangs at 99% indefinitely on these appliances. For recovery-rollback purposes, **non-quiesced is acceptable** — Photon + the inner K8s cluster crash-recover cleanly on revert. See `New-Snapshot -Quiesce:$false` in §15.1 below.

---

15. Incident Log — 2026-05-17 Recovery After Cert Rotation

Recurrence of the half-broken Fleet/vcops integration, this time triggered by a Fleet certificate rotation rather than instance-limit deadlock. Resolved cleanly via the **KB 402063** API path. **No Phase 5 (Postgres direct write) was required** — the API path executed cleanly because the trigger was a clean external state change (cert rotation) rather than a stuck `VropsDeleteCapabilitiesTask` with null `vropsMasterNodeIP`.

15.1 Trigger

On 2026-05-16 16:48 UTC, Fleet's `/opt/vmware/vlcm/cert/server.crt` and `server.key` were rotated to introduce a multi-SAN cert covering `automation-node1.lab.local`, `vcfa-mgmt-0.lab.local`, `lcm.lab.local`, `fleet.lab.local` and IPs `.91`, `.92`, `.95`. New SHA-1 thumbprint: `49:B3:F9:95:AA:26:8F:F1:E2:EE:CC:B4:3D:2B:C3:10:0D:D2:D5:E0`, valid through 2028-05-15. **VCF Operations was not in-band notified** of the swap; its Java truststore retained the prior thumbprint and could not validate the new chain on its next call to Fleet.

15.2 Symptom Signature

Surface	Observation
vcops UI → Fleet Management → Lifecycle	`VCF Operations Fleet Management is Not Ready`
vcops UI → Admin → System Status	`Fleet Management: Connected` (green) — misleading; admin link alive, plugin trust broken
`/storage/log/vcops/log/web-vcf.log`	`Caused by: sun.security.validator.ValidatorException: PKIX path building failed: sun.security.provider.certpath.SunCertPathBuilderException: unable to find valid certification path to requested target`
`/storage/log/vcops/log/product-ui/localhost_acc`	Repeating `GET /vcf-operations/plug/ops-lcm/lcm/bootstrap/api/status HTTP/2.0 400` every 5s

Surface	Observation
ess_log	
Fleet UI → Components	`operations` row, status `New Deployment`, **Delete option greyed out**
Fleet API	`GET /lcm/lcops/api/v2/environments` returned env `888501ce-eb95-4ce8-8043-ab216906bc67`, status `COMPLETED`

15.3 What Did Not Work (and why)

Action tried	Result	Why it failed
Reboot Fleet appliance per KB 429291	Pods returned to 27/27 Running in ~5 min; symptom unchanged	KB 429291 addresses the *sync-state* root cause; the *trust-state* root cause caused by cert rotation persists across reboot
`REPLACE` button in vcops Admin UI	`Error connection Fleet management node, try again`	Stale 1:1 record on Fleet blocked the new registration (KB 404388) and the failed attempt left a second partial entry
Fleet UI → Components → 3-dots → Delete	Greyed out, no tooltip	9.0.1 architecture: when only one integrated component exists, the UI offers no path to remove the last one
Fleet UI → Components → Manage → "..." top toolbar	Menu contains Scale Up / NTP / Patches / Snapshot / Logs — **no Delete**	Same 9.0.1 UI constraint

15.4 Pre-Recovery Snapshots

```
 # Run from Windows workstation
Set-PowerCLIConfiguration -InvalidCertificateAction Ignore -Confirm:$false -Scope Session | Out-Null
$cred = New-Object System.Management.Automation.PSCredential('administrator@vsphere.local',
        (ConvertTo-SecureString '<your-password>' -AsPlainText -Force))
Connect-VIServer -Server 192.168.1.69 -Credential $cred -Force

New-Snapshot -VM 'fleet'     -Name 'pre-KB430561-disconnect-2026-05-17' `
             -Memory:$false -Quiesce:$false -Confirm:$false
New-Snapshot -VM 'vcf-ops'   -Name 'pre-KB402063-API-DELETE-2026-05-17' `
             -Memory:$false -Quiesce:$false -Confirm:$false
New-Snapshot -VM 'collector' -Name 'pre-KB402063-API-DELETE-2026-05-17' `
             -Memory:$false -Quiesce:$false -Confirm:$false
```

Quiesce note. The first Fleet snapshot used `-Quiesce:$true` and stuck at 99% for 13 min before the PowerCLI connection aborted. The guest-side VMware Tools quiesce hook deadlocks on the file locks held by the `kind-bootstrap-control-plane` container. For recovery rollback, `Quiesce:$false` is the right choice (see Lesson 11).

15.5 Disconnect via vcops Admin UI

Per the main §6.2 procedure. After clicking Disconnect, the System Status panel transitioned from `Connected` + `REPLACE` / `DISCONNECT` buttons to the empty/ `Connect` state. The stale env record on the Fleet side remained, blocking subsequent reconnect attempts — that is what §15.6 removes.

15.6 Enable Fleet Break-Glass UI

```
# Run on the appliance only — Fleet (192.168.1.95) as root
touch /var/lib/vrlcm/UI_ENABLED
ls -la /var/lib/vrlcm/UI_ENABLED
# -rw-r----- 1 root root 0 May 17 16:34 /var/lib/vrlcm/UI_ENABLED
```

VCF 9.0.1's Fleet appliance runs a hybrid stack: the classic `vrlcm-server.service` (Java on port 8080, `vmlcm-service-9.0.1.0-SNAPSHOT.jar`) **plus** a KIND-based `kind-bootstrap-control-plane` Docker container hosting 27 `vmsp-platform` pods. The `UI_ENABLED` flag re-exposes the classic Java LCM UI — intentionally hidden in 9.0+ but retained for break-glass.

15.7 KB 402063 API DELETE

```
# Run on Fleet (.95) OR any host with curl + admin@local credential
ENV=$(curl -sk -u admin@local:'<password>' \
  https://localhost/lcm/lcops/api/v2/environments \
  | python3 -c "import sys,json; print(json.load(sys.stdin)[0]['environmentId'])")
echo "$ENV"
# 888501ce-eb95-4ce8-8043-ab216906bc67

curl -sk -u admin@local:'<password>' \
  -X DELETE \
  -H "Content-Type: application/json" \
  -d '{
    "controllerType": "string",
    "deleteFromInventory": true,
    "deleteFromVcenter": false,
    "deleteLbFromSddc": true,
    "deleteWindowsVMs": true
  }' \
  "https://localhost/lcm/lcops/api/environments/$ENV/delete"
# {"requestId":"1772209a-f300-481d-86ae-ce489090f9b7"}
```

`deleteFromVcenter: false` is load-bearing. Without this flag (or with `true`), Fleet will attempt to power off and delete the `vcf-ops` and `collector` VMs from vCenter. With `false`, only the LCM DB record is cleared; the VMs are left untouched. Confirm before pressing Execute.

The `"controllerType": "string"` field is a Swagger placeholder retained verbatim from KB 402063; Fleet's `RequestProcessor` ignores it. Substituting `"VROPS"` or other product strings is not required.

15.8 Verify Completion

```
# Run on Fleet — tail the LCM log for the requestId
tail -F /var/log/vrlcm/vmware_vrlcm.log | grep -E "1772209a|environmentdelete"
```

Success log line (verbatim, this incident):

```
2026-05-17T17:12:46.450Z INFO vrlcm[1244] [scheduling-1] [c.v.v.l.r.c.RequestProcessor]
  -- Updating the Environment request status to COMPLETED for request ID :
  1772209a-f300-481d-86ae-ce489090f9b7 with request type : DELETE_ENVIRONMENT.
```

Post-DELETE verification matrix:

Check	Result
`GET /lcm/lcops/api/v2/environments`	Count `0`
`GET /lcm/lcops/api/v2/environments/888501ce-...`	HTTP `404`
`vcf-ops` VM	PoweredOn, Heartbeat Running, IP 192.168.1.77 ✓
`collector` VM	PoweredOn, Heartbeat Running, IP 192.168.1.79 ✓
`fleet` VM	PoweredOn, Heartbeat Running, IP 192.168.1.95 ✓

> **API quirk to ignore.** `GET /lcm/api/v2/requests/{requestId}` returns `HTTP 500 Internal Server Error` against a body like `{"timestamp":...,"status":500,"error":"Internal Server Error"}`. The DELETE itself succeeded; only the request-lookup endpoint is broken on this build. Use the LCM log (`/var/log/vrlcm/vmware_vrlcm.log`) for status, not the API.

15.9 Reconnect via vcops Admin UI

Field values used (verbatim):

Field	Value
Fleet FQDN	`lcm.lab.local`
Username	`admin@local`
Password	`<Fleet admin@local password>`
Cert thumbprint	`49:B3:F9:95:AA:26:8F:F1:E2:EE:CC:B4:3D:2B:C3:10:0D:D2:D5:E0` (verified live before accept)

Result: VCF Ops Admin UI returned to `Connected`; Fleet Management → Lifecycle → VCF Management cleared the `Not Ready` banner.

15.10 Cleanup

```
# Restore default state — disable break-glass UI
rm /var/lib/vrlcm/UI_ENABLED
```

Consolidate the chain of snapshots taken in §15.4 once Fleet operations stabilize (do not leave the 9 GB chain on each appliance long-term).

15.11 KB Citation Map (Resolution Order)

KB	Used	Notes
402063	**Authoritative DELETE endpoint and payload**	Public; cited verbatim in §15.7
410260	Same procedure, support-restricted	Broadcom-Support-only; not needed when 402063 suffices
430561	Procedure outline (disconnect → enable UI → delete → reconnect)	Public; predates 9.0.1's UI greyout

KB	Used	Notes
432320	Root-cause framing: 1:1 limit + cert change	Public; useful for symptom ↔ cause mapping
429291	Tried first; insufficient for cert-rotation case	Public; reboot fixes sync state, not trust state
404388	Explains `REPLACE` button error	Public
433975	Cert SAN issue	Public; ruled out (SANs were correct on the new cert)

15.12 Time Breakdown

Phase	Elapsed
Symptom recognition to first wrong attempt (KB 429291 reboot)	~30 min
Reboot wait + reconfirm failure	~15 min
Disconnect + Fleet UI exploration (Components → greyed Delete)	~30 min
Snapshot phase (incl. failed quiesced attempt)	~50 min
KB 402063 identification + API DELETE + verification	~5 min
Reconnect via vcops Admin UI	~2 min
Total	**~2.5 hours** — net hands-on once correct KB identified: **~15 min**

Revision History

Version	Date	Notes
1.0	April 26, 2026	Initial release covering half-broken Fleet/vcops integration RCA, instance-limit deadlock, KB-aligned recovery procedure (Phases 1-7), full SQL cleanup script, Fleet API reference, FK tree, and lessons learned.
1.1	May 7, 2026	Option B PowerShell treatment: added top-of-doc callout linking to `../POWERSHELL-TRANSLATION-REFERENCE.md`; added paste-ready PowerShell sibling blocks for workstation-runnable bash blocks (Fleet `/lcm/lcops/api/v2/environments` GET + DELETE via `Invoke-RestMethod` with Basic-auth header, Phase 5.3 instance-limit log grep, Phase 5.4 DNS/reachability probe with `Resolve-DnsName` + `Invoke-WebRequest`, Phase 4 DELETE verification log grep, Phase 5.4 NPE-flood ERROR-count check, Phase 7 fresh-env verification, Phase 7 bootstrap SUCCESS grep). PS 5.1 `ServicePointManager` cert-bypass + TLS 1.2 header inserted once at the first PS sibling. Appliance-only blocks (`/etc/systemd/system/` drop-in, fleet `systemctl is-active`, `touch` / `rm` `/var/lib/vrlcm/UI_ENABLED`, in-tree `psql` heredoc, `psql -f`) flagged "Run on the appliance only" with no PS sibling. No content/procedure changes.
1.2	May 17, 2026	Added §15 Incident Log for 2026-05-17 recurrence triggered by Fleet certificate rotation (new thumbprint `49:B3:F9:95:AA:26:8F:F1:E2:EE:CC:B4:3D:2B:C3:10:0D:D2:D5:E0`, multi-SAN covering automation-node1, vcfa-mgmt-0, lcm, fleet). Resolved via KB 402063 API DELETE on `/lcm/lcops/api/environments/{envId}/delete` with `deleteFromVcenter: false`; recovery completed in ~15 min net hands-on

Version	Date	Notes
		without Phase 5 DB cleanup. Added Lessons 9-11 covering (a) KB 402063 as the public companion to restricted KB 410260, (b) the greyed-out UI Delete in 9.0.1 Fleet, (c) quiesce-deadlock on K8s/Docker-heavy guests. Documented the failed paths (KB 429291 reboot, REPLACE button, UI Delete) and why each was insufficient for the cert-rotation root cause.

Document End

See also: - `VCF-Automation-Deployment-Troubleshooting-Report.md` — for VCF Automation deploy issues that surface AFTER this integration is healthy - `NSX-Manager-Cold-Start-Troubleshooting-Report.md` — for NSX Manager service recovery after similar cold-start failures - `VCF_Operations_Full_Recovery_RCA.md` — for complete vcf-ops appliance recovery